



Conceptual Data Modeling Patterns: Representation and Validation

Dinesh Batra, Florida International University, USA

ABSTRACT

The tremendous demand for software productivity has led to the idea of reuse of solutions that have worked successfully in the past. The notion of a design pattern is now well accepted in software design, and research in the area of data modeling has also begun. Although two books have explicitly attempted to cover this area, the representations provided in the books seem to be focused on specific applications and do not provide a generic and comprehensive set of templates. Another book attempts to address the problem but provides patterns at a level of granularity too small to be useful. This paper teases out underlying structures that tend to occur frequently in these books and provides patterns at an abstract and more useful level of granularity. It describes 11 data modeling patterns commonly found in business scenarios. The patterns are then validated by checking the frequency of occurrence of each pattern in the data representations included in three comprehensive texts of reference models. Two of these sources are targeted mainly at practitioners, and the third is academic oriented and targeted at students learning data modeling. Results indicate that although certain patterns are used more frequently than others, most of the 11 structures occur with adequate frequency to qualify as patterns. A comparison reveals that the frequency distribution of patterns is different among these sources. Further, the academic-oriented source distinctly focuses on different patterns as compared to the other two sources. The paper discusses the differences and provides specific recommendations on improving pedagogy in conceptual data modeling.

Keywords: data classes; database conceptual design; entity diagrams; heuristic development; unified modeling language

INTRODUCTION

One thing expert designers know *not* to do is to solve every problem from first principles (Chi, Glaser & Farr, 1988; Gamma, Helm, Johnson & Vlissides, 1994). Tremendous demand for software productivity has led to the idea of reuse of solutions that have successfully worked in the

past. Although the notion of a design pattern is now well accepted in software design, work in the area of conceptual data modeling has just started. The purpose of this paper is to foster this progress by providing patterns for conceptual data modeling at a level of abstraction and granularity that encourage pattern recognition and reuse. The patterns are subjected to valida-

tion by determining the frequency of their occurrence in data models found in comprehensive texts of reference models.

Gamma et al. (1994), whose work on patterns pertains to object modeling in software design, have defined a design pattern as a description of communicating objects and classes that are customized to solve a general design problem in a particular context. In the context of data modeling, a pattern can be defined as a description of objects (or entities), relationships, and attributes that are customized to solve a general conceptual or logical database design problem. Gamma et al. specify four essential elements of a pattern—Pattern name, Problem, Solution, and Consequences. The *pattern name* is a handle to describe a design problem, its solution, and consequences in a few words. The *problem* describes when to apply the pattern. In a data-modeling context, the *solution* would describe the conceptual and/or logical design of the pattern. The *consequences* are the results and trade-offs of applying the pattern. In this paper, we propose *variation* as a fifth element of a pattern. The *variation* of a pattern describes a limited deviation to the basic structure of the pattern. A variation of a pattern must bear resemblance to both the problem and the prescribed solution for the normative form of the pattern. Naming the variation provides us with a richer vocabulary of templates and alternatives within a given scenario.

Pattern recognition is a well-researched topic in cognitive psychology. Almost three decades ago, Reynolds and Flagg (1977) devoted a complete chapter to pattern recognition and discussed research in the area of speech and visual recognition. They state:

Template-matching is the simplest process by which pattern recognition can occur.

According to this theory a large number of internal representations, or templates, of various objects are stored within the permanent, or long-term memory. Associated with each representation is a label or meaning. When an external stimulus is presented it is compared to the various internal templates until a match is found. (p. 61)

The main premise in our paper is that a limited number of basic templates can cover a large number of data modeling scenarios.

The theoretical basis of data modeling patterns can also be expressed using three notions: production/schema, analogy, and abstraction. Production system theories are based on the claim that underlying human cognition is a set of condition-action pairs called productions (Anderson, 1996). The conditions specify some data patterns, and if elements matching these patterns are in memory, then the productions can apply. Analogy is a common problem-solving approach. With this method, the problem solver attempts to use the structure of the solution of one problem to guide solutions to other problems (Anderson, 1985; Metsker, 2002). For example, a designer may note that an order form and the invoice generated have a similar structure, and the data model of the order can be employed to model invoice by using a “transfer in problem solving” (Kotovsky & Fallside, 1989) approach. This is feasible because order and invoice are examples of a deeper structure called transaction. Abstraction facilitates defining the deeper structure of a pattern (Greeno, 1989). Any template structure is based on the notion of abstraction. Abstraction achieves parsimony in the number of structures to be recalled but relies on designer skill to accurately map an application description into a deeper structure. This mapping is depen-

dent on analogical processes that may be activated by surface and structural similarities.

Patterns can be successfully employed in conceptual data modeling (CDM), which can be considered as a problem-solving activity (Batra & Wishart, 2004). Problem solving is a procedure in which applicable declarative knowledge (defined as the problem space) is used to move from the initial representation of the problem to the desired goal state (representation transformation). The basic process that occurs in problem solving is a search of the problem space for the correct operators (chunks of information) to create the goal representation (Reeves, 1996). There are four ways of enhancing the search (Anderson, 1985; Newell & Simon, 1972):

1. Following algorithms, or specific steps that are known to lead to the solution or goal representation under specific circumstances.
2. Following heuristics, or general steps that help to activate memory that has a high probability of generating a solution.
3. Following creative techniques that help a person represent the problem in a way that stimulates existing operators in new ways.
4. Increasing the probability of finding relevant declarative knowledge. The more information present, within reason, the more likely that the search for operators will be successful.

When using patterns, conceptual data modeling employs the second and the third approach. Pattern recognition reduces the effort of processing facts individually and speeds up understanding or the generalization of insight (Reeves, 1996). The approach is certainly heuristics based since it is dependent on semantic structures frequently found in business applications. Fur-

ther, the approach is novel in which the modeling unit is not an entity or a relationship but a pattern, consisting of several entities and relationships. By providing semantic units of modeling, one could argue that the probability of finding declarative knowledge is also increased.

Thus, pattern recognition can be considered as a conceptual modeling technique. Gemino and Wand (2004) argue that conceptual modeling techniques are similar to ontologies (Weber, 2003). Like an ontology, a new conceptual modeling technique requires initial constructs and structures. The success of an ontology or a conceptual data modeling technique depends on empirical verification, which strengthens the acceptance of proposed structures.

LITERATURE SURVEY

A literature survey reveals that there have been several attempts at listing data modeling patterns. Three notable books are from Hay (1996), Fowler (1997), and Coad (1997). Hay (1996) does a fine job of introducing each pattern and building a chapter around it. Specifically, the chapter on contracts (pp. 95-116) provides a template for transactions. However, the patterns are of larger granularity and actually contain several patterns. For example, Hay's (1996) chapter on procedure and activities is built around a pattern (see p. 70), which also contains a recursive structure. Since the recursive structure is a pattern in itself, it would be better to divide the pattern into two elements—plan and recursion. The advantage of this decomposition is increased reuse. A larger pattern can lead to lower modeler performance (Batra & Wishart, 2004). For example, if a procedure is composed of activities such that an activity does not recursively decompose into

smaller activities, a larger pattern is unnecessary and may actually lead to worse performance. Thus, we should have patterns of smaller granularity.

However, is there a hazard in decomposing patterns to a granularity too fine? Coad (1997) provides 30 object modeling patterns at a very high level of abstraction and a very fine level of granularity. A high level of abstraction implies that the patterns are not too domain specific. A very fine level of granularity means that the patterns are small—typically with only two entities and one relationship. The application of these patterns can be cumbersome since a large number of patterns are required for even simple cases. In fact, Coad's patterns may be classified into four groups: transaction, aggregate, plan, and interaction. Overall, this list does not adequately address the patterns in business applications.

The challenge then is to provide a limited number of patterns at a fairly high level of abstraction and an appropriate level of granularity with reasonably domain-independent structures having a high potential for reuse. Among these criteria, the one that is most difficult to elucidate is the level of granularity. What may be an appropriate level? An analogy may clarify this issue. A component (whether engineering or software) is generally useful when it is neither at a high level of granularity, nor at a low level of granularity. A nut or a bolt by itself can be used in numerous applications but does not provide large increases in productivity. However, when combined with other parts, such parts can be used to develop components, such as power supply, switches, and adapters, which can then be used to quickly put together different systems. Conversely, if the component is composed of too many parts, it becomes too large and not very reusable. It is difficult to make a hard and fast rule about the granu-

larity of the components; however, in general, a component that has a very fine level of granularity does not significantly increase productivity, and a component that has a large level of granularity does not facilitate reuse.

This paper provides representation of common templates and follows it up with validation of the structures. It is assumed that the reader is familiar with the notions of entity and relationship (Chen, 1976) and is also familiar with UML (George, Batra, Valacich & Hoffer, 2004; Scott, 2001; Shah & Slaughter, 2002). First, the patterns are listed. These are usually based on three main sources—Hay, Fowler, and Coad—listed in this section but are improvised to optimize recognition and reuse. The first step relies on judgment and experience and may be considered tentative, so it is then subjected to the second step, which demands validation. To achieve validation, the patterns are evaluated based on frequency of occurrence in data models from sources that are comprehensive and address enterprise business areas, such as logistics, accounting, and human resources. The proposed structures would be considered validated if each pattern has adequate frequency, and there are no discernible patterns in the residual after the proposed structures are teased out.

REPRESENTATION OF DATA MODELING PATTERNS

To illustrate the patterns, this paper employs the UML notation for showing entity classes and the relationships among them. Note that the entity class is one of the most important stereotypes of a class (Rosenberg, 1999) and resembles the entity (or entity type) concept of the entity-relationship (ER) model (Chen, 1976). We

use the terms entity, entity type, and entity class interchangeably. The primary key attributes are stereotyped (Rumbaugh, Jacobson & Booch, 1999) to include the keyword <<Primary>>. In certain cases, the relational representation (Codd, 1970) is included for clarification. For the same reason, if a foreign key is included in the UML representation, the attribute is stereotyped using the keyword <<Foreign>>. A multivalued attribute is stereotyped as <<Multivalued>>.

Pattern 1: Entity Event Pattern

A common example of an entity type is Employee. In textbooks, it is common to see the representation in Figure 1a.

It seems that an employee is a single “thing” comprised of certain characteristics, thus, an entity. A closer examination will reveal that this entity is made up of attributes that do not necessarily group together. For example, consider a person Mary who is an employee at a company. It is unlikely that her date of birth will ever be changed. She may get married and decide to change her last name. Depending on the application, you may or may not maintain the previous last name; the last name can simply be replaced by the new name since the identifying attribute is empNo. (If you do, then you will need to record the date the change was made.) Although last names do not change frequently, addresses do change often. An application may require that the past addresses be maintained. In this case, you need to also maintain the start and stop dates of the addresses of the employee (Figure 1b). Each change is an event, and you may need to store a certain number of past events (e.g., last three addresses or addresses within the last five years) pertaining to the employee.

The pattern entity event is not explicitly mentioned in any of the books covering data modeling patterns; however, there are examples in the Hay (1996) and Fowler (1997) books that suggest that this structure could occur occasionally. Cases from books like Whitlock, Nelson, and Papp (2003) reveal that this structure would be found quite frequently.

Another reason why this pattern should be considered is that, with a slight extension, it can be used to capture history (e.g., product history, customer history, employee history, etc.). If a number of past events need to be stored, the result is a history of some characteristics of the entity. Each entity may have several histories, for example, salary history, position history, and so forth. A history consists of a chronological list of events. There are at least two variations of history. One is based on periods of time. A period of time is also called an interval (or sometimes a chronon). Thus, Mary’s salary may be deposited bi-weekly, and her performance goals may be set at regular intervals, say, monthly. However, her job history, that is, her promotions since joining the company, might not have come at regular intervals and can be represented by specifying the start and end dates of her position.

Thus, at least three entity variations may need to be stored (see Figure 1b):

1. The entity with attributes that do not normally change over time (e.g., the

Figure 1a. Entity class employee

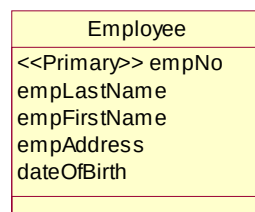
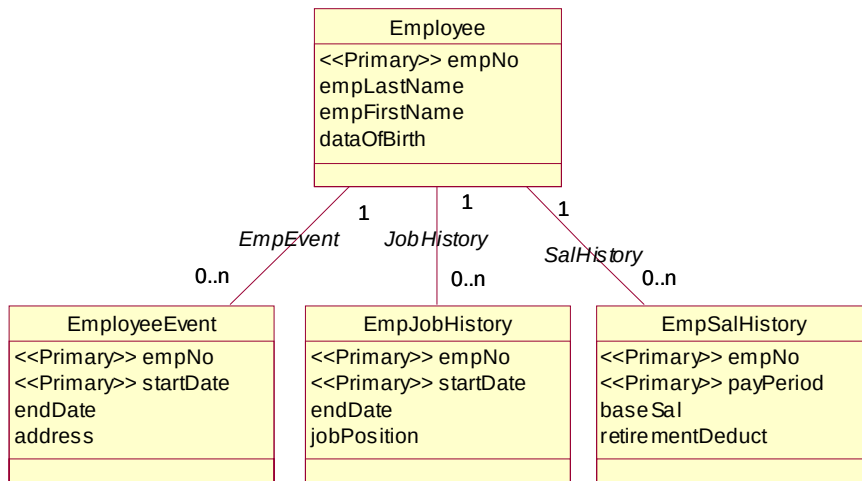


Figure 1b. Entity event pattern using employee for illustration



entity Employee). No record needs to be maintained for old values if an attribute value does change.

2. The entity with attributes that may change over time (e.g., EmployeeEvent). Each change is an event, and the past values and the respective time stamps need to be maintained.
3. Entity history represented using intervals (e.g., EmpJobHistory), or start and end dates (e.g., EmpSalHistory).

The logical representation of an entity depends on the data model that is used. The relational representation, which requires normalization, may require several relations. However, the object-relational representation, which allows composite and multivalued attributes, may require fewer object-relations; for example, EmployeeEvent can be nested in Employee.

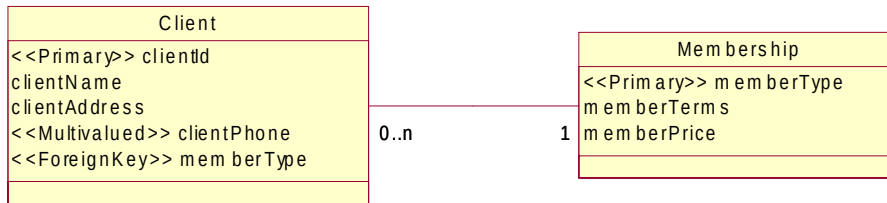
Pattern 2: Entity Type

It is common to classify entity into entity types (Figure 2). Thus, members may

be classified by membership type, products may be classified into product lines, and vehicles may be classified by vehicle make and model, and so on. One advantage of typing is that decision makers at higher levels in organizations can obtain aggregate information for maintaining control. Thus, donors may be classified into individuals, corporations, and foundations. A vice president, for example, would typically like to know how the total donations are divided among the three groups. Such information can be very useful from decision making, marketing, and accounting. Similarly, clients may purchase a membership type, which allows them certain benefits (Figure 2).

The entity type pattern is elegantly described in Hay's (1996) chapter "Things of the Enterprise" as well as in Fowler's (1997) opening chapter "Accountability". Note that the entity type pattern is different from the subtyping/supertyping notion. In the subtype relationship, which is also called "is-a" relationship, the primary keys of the two entities are the same; in the entity type pattern, they are different. The

Figure 2. Type pattern example



type pattern is modeled by a conventional relationship using foreign keys.

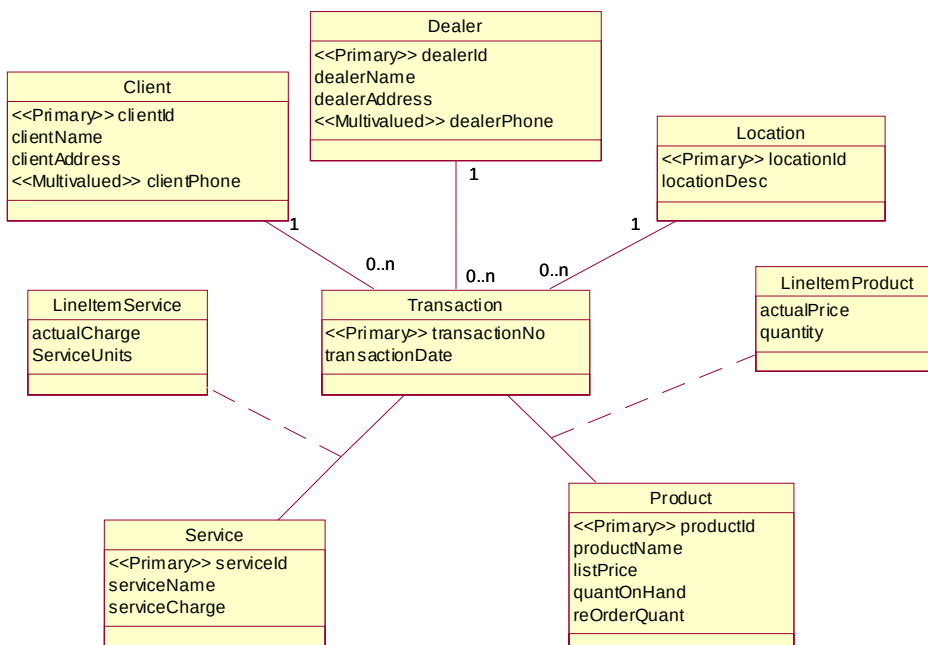
Pattern 3: Generic Transaction

The transaction is perhaps the most important found pattern (Figure 3). Hay (1996) provides an excellent treatise of the pattern in the chapter “Contracts”. Fowler (1997) covers it implicitly under applications like trading. The notion of transaction lies at the core of business applications. A

transaction is usually given a number or code for identification. It is generally related to certain other entities, and if the product entity pertains to a generic product, the resulting data modeling structure is called the generic transaction pattern.

Examples of transactions include purchase order, sales invoice, restaurant order, reservation, rental, registration, delivery note, deposit, and so forth. Central to a business transaction is the transaction entity, which generally has one-many (1:n)

Figure 3. Generic transaction pattern



relationships with each of several party records. Typical examples of party records are client, customer, and so on, and transaction-handling agents like salesperson, waiter, telemarketer, agent, and so forth. Another important characteristics of a transaction is the location or the place the transaction occurred. Typical examples are region, register, workstation, and so on.

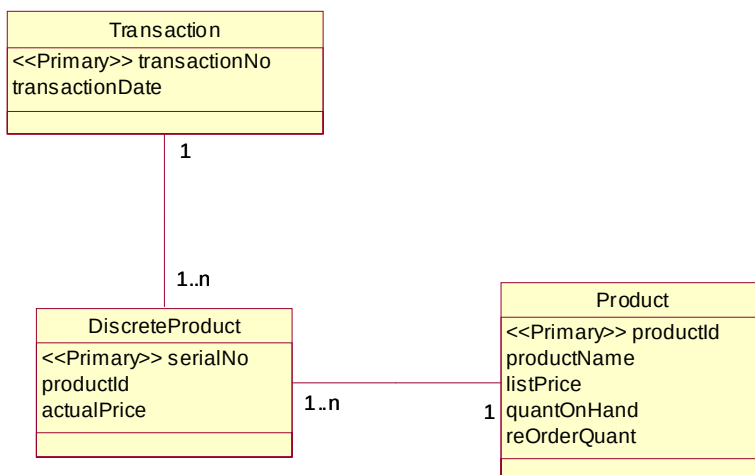
Figure 3 shows a generic transaction, which is the one shown most often in books illustrating reference models. A generic transaction usually has a many-to-many (m:n) relationship between transaction and product, and/or transaction and service. These line item relationships are shown as associative entities in UML. Examples of items are products, services, dishes ordered, checks deposited, items delivered, rooms reserved, and so forth. Note that a product here pertains to a generic product, not a specific product that is identified by a serial number. Thus, a table is likely to be identified only by its product code; in this case, the sale will lead to a generic transaction.

Pattern 4: Transaction for Discrete Product

This is a variation from the generic transaction pattern. A product is usually identified by a product number, which distinguishes the kind of product. When the specific instance of the product, called discrete product, needs to be identified, it is conventional to use a serial number instead of a product number. For example, since ownership is important for products like vehicles, airplanes, boats, computers, and so on, an individual product is identified by a serial number. These products will also have a product number; however, the transaction is related with the discrete product. A product can be associated with several discrete products, while a discrete product is associated with only one product.

The LineItem relationship between Transaction and DiscreteProduct is usually 1:n with discrete product on the many side with respect to the transaction (Figure 4). This would be the case if a shop sells new products only, and several products can be

Figure 4. Discrete product pattern



included in the transaction. It is assumed that a discrete new product can be sold only once; if a used discrete product is sold and the transactions record of its previous sale is also present in the active database, the transaction record is deleted and migrated to a historical database.

However, this relationship itself has a number of variations. The LineItem relationship between Transaction and DiscreteProduct can show any of several multiplicities when a discrete product is involved. If new and used products are both sold, and all transaction records are to be maintained, the relationship can be m:n, since a discrete product can be sold more than once. If a product is expensive with many options and add-ons, it may be appropriate to limit one discrete product per transaction. If only new discrete products are being sold (e.g., airplanes), the LineItem

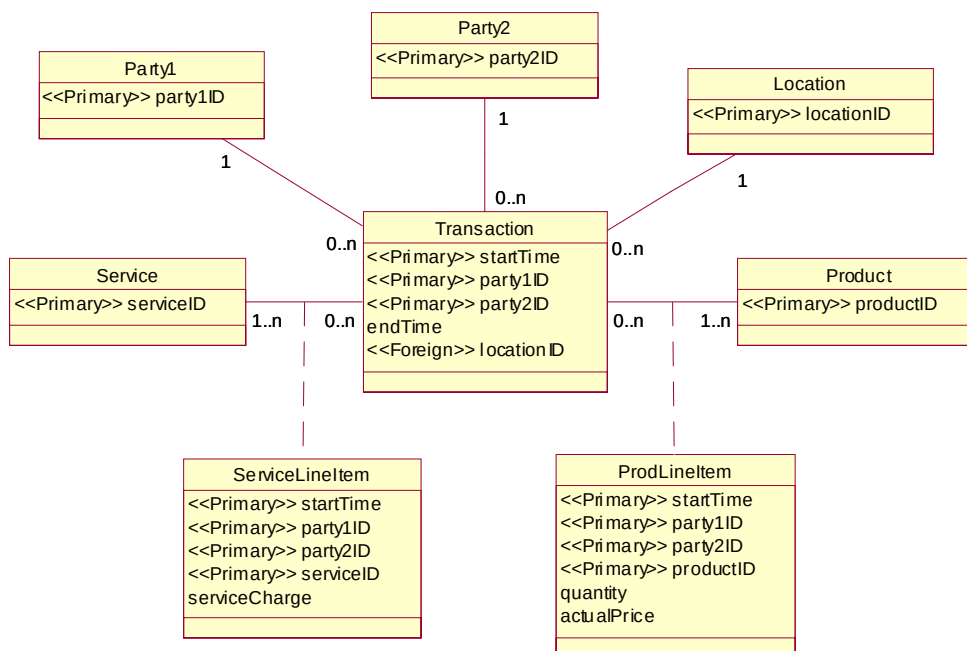
relationship is 1:1. However, if used discrete products are sold (e.g., used cars), such that a transaction involves only one discrete product, then the relationship can be 1:n where the many side is the transaction entity. Thus, in special cases, the multiplicity deviates from the norm.

Hay (1996) presents the distinction between a generic and a specific product but does not discuss the variations because of different multiplicities that can result in the case of the specific product. Books generally do not discuss the effect on multiplicity when there is a change in the scenario from the generic to the discrete transaction.

Pattern 5: Time-Based Transactions

This is another variation from the generic transaction pattern. Generally, a

Figure 5a. Typical configuration of time-based transaction pattern



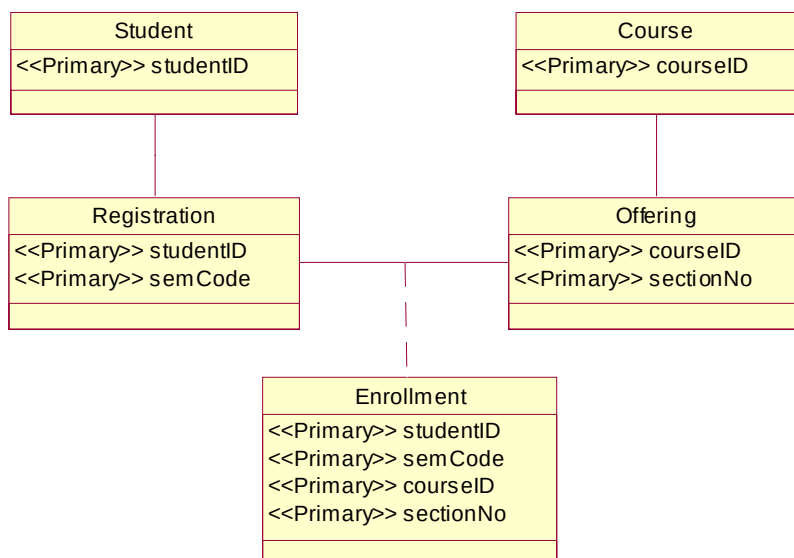
transaction is given its own attribute, like transaction number, to serve as a primary key. Using a system-generated sequence of numbers can generate the primary key. In certain cases, a time stamp along with primary keys of master, place, and product entities can be concatenated to identify the transaction. Scheer (1998) represents all transactions using time elements, although he admits this is more for the sake of clarity and that alternative ways of representing transactions are equally valid. Figure 5a shows a typical representation shown with minimum attributes. The time-based transaction pattern resembles the generic pattern in the overall structure but differs in certain details.

Although, a time-based transaction can also be shown as a relationship between several entities and time, it is more elegant to show it as an entity so long as the primary key is correctly defined. Time is usually represented by *startTime* and

endTime, or by an interval, and always plays a critical role in determining the primary key. However, the primary key of a time-based transaction may be (*startTime*, *party1ID*), (*LocationId*, *startTime*), or a variation, and the UML representation needs to display the appropriate primary key. Instead of *startTime*, the primary key may involve a period of time, that is, an interval such as hour, day, week, month, or year.

A registration scenario can provide an example of a time-based transaction (although it is better handled by a generic transaction). Registration is the transaction; Enrollment is the LineItem; *semCode* is the interval attribute; *StudentNo* is the primary key of a master entity (Student); *CourseID* is the primary key of a product entity (Course); and *SectionNo* is a sequence number to distinguish offerings of the same course (Figure 5b). The Registration identifier includes a time element *semCode*.

Figure 5b. Registration as a time-based transaction



Note that the primary key of Enrollment has four attributes; this is the main disadvantage of this representation. The advantage is that the primary key can enforce certain business rules; for example, a student can register only once in a semester, a student can enroll only once in a section in a semester, and so forth.

Pattern 6: Subsequent Transaction

Coad (1997) specifies subsequent transaction as a pattern. However, it is also a mechanism to help an analyst think about what transactions relate to the one being modeled. Data flow of a typical company reveals that one transaction typically leads to another. Thus, an inquiry can lead to quotation, which can lead to a sales order, which in turn can lead to sales invoice. Checkout of inventory items involved in sales invoice can lead to purchase order which can lead to purchase invoice, and so on.

If the subsequent transaction requires only a change of status, this can be addressed by a Status attribute. However, if a transaction can spawn another transaction, this needs to be modeled by a relationship, which usually has the 1:n multiplicity. For example, if an order can lead to several invoices, then the relationship is 1:n, since an invoice usually pertains to one order (Figure 6). However, if an invoice can pool shipments for various orders, then the relationship is m:n, an example of variation on the normal transaction-subsequent transaction.

The subsequent transaction pattern can be discerned if workflow models of the domain are available. The analyst can examine the flow of processes and examine how the transactions are related.

Pattern 7: Transaction Summary for Data Warehouse

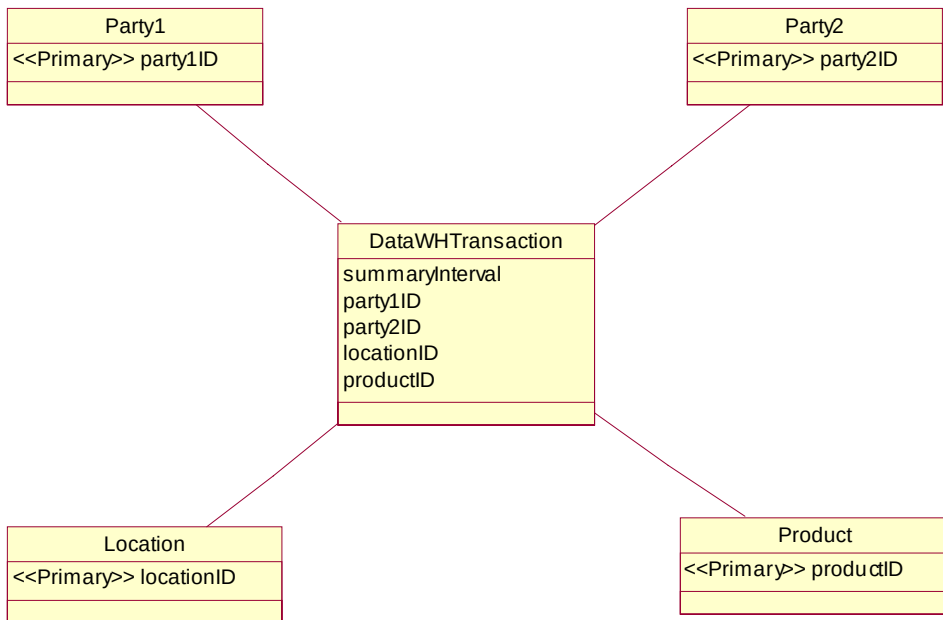
This pattern is somewhat similar to the time-based transaction pattern since the transaction entity has no intrinsic primary key (Figure 7); instead, it is based on a time interval concatenated with the primary keys of the master entities, such as Customer, Product, and Salesperson. The data warehouse structure is discussed in several books, for example, Gray and Watson (1998). The time element is usually a larger interval—transactions may be aggregated by day, week, or month. In the figure, the time period attribute has been named intervalSummary since warehouse data is usually summarized. Since data is not added to the DataWHTransaction entity through conventional data entry, the entity need not have a primary key. Line item data is not shown in the pattern since this is incorporated in the DataWHTransaction entity during summarizing. There are one or more party entities, and the configuration appears like a star.

The attribute summaryInterval pertains to the granularity of the time period. However, this can be replaced by start date and end date. One can expect that the higher the level of the manager, the longer

Figure 6. Subsequent transaction



Figure 7. Data warehouse pattern



the time period for the transaction summary; it could be a day for a supervisor and month for a vice president.

Since data warehouse data is static, it is likely that the DataDWTransaction entity is denormalized. Consider a transaction relation OrderSummary that is obtained by running queries against transaction data. It may violate normal forms by including data about customers, salespersons, and products:

OrderSummary(Month, CustomerNo,
SalespersonNo, ProductNo,
CustomerName, SalespersonName,
ProductLine, ProductDesc,
ProductLineCode, Amount, Quantity)

Pattern 8: Recursion Pattern

The recursion pattern is illustrated in practically every book on data modeling

patterns. A recursion pattern indicates a unary relationship (i.e., relationship among instances of the same entity) that can be exploded to several levels. For example, the Bill of Materials example, so commonly found in reference models, depicts a recursive m:n relationship between instances of the same entity (Figure 8a). This explosion is multilevel and is common in manufacturing operations. In this case, a part can have subparts, each of which also is a part, which can be recursively exploded into subparts. Similarly, a prerequisite relationship, which depicts courses that are a prerequisite to other courses, is a recursive m:n relationship.

A unary one-one relationship cannot be recursive. However, there are several examples of recursive 1:n relationships. For example, a job position can be supervisory to several job positions (Figure 8b). If a job position can report to more than one posi-

Figure 8a. Bill of materials as an illustration of recursion

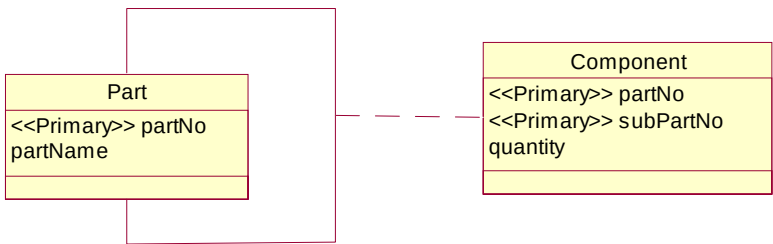
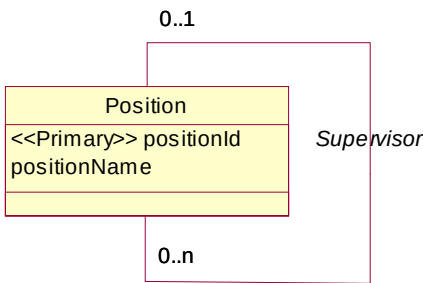


Figure 8b. A recursive one-many relationship



tion, then the organization has a matrix structure and the data modeling relationship is unary m:n.

Pattern 9: Strict Hierarchy Pattern

A strict hierarchy implies a tree-like structure where an instance of parent entity can be linked with several instances of a child entity, but an instance of child entity links to only one instance of parent entity, and both parent and child entities have similar characteristics. Note that the recursive 1:n structure is among instances of the *same* entity while the hierarchic structure is defined as among instances of *different* entities. The number of levels are fixed and known in the case of hierarchy, but unknown in the case of recursion. However, if the parent and child entities have identical characteristics, an analyst may choose the recursive structure even if the number

of levels are known. Hay (1996) elegantly compares these structures in one of the opening chapters of his book. Other data modeling books do not explicitly discuss this structure. It is left to the validation process to confirm or disconfirm this pattern.

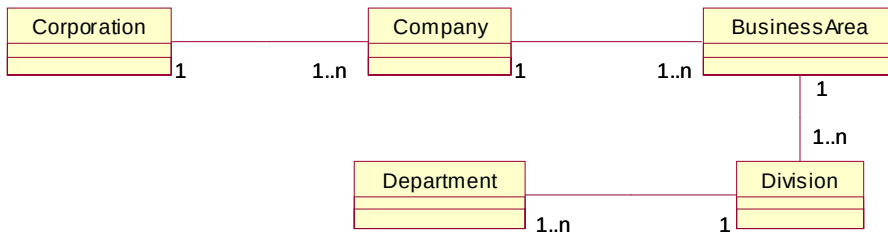
A strict hierarchic structure is represented by a 1:n relationship, since, by definition, hierarchy generally implies a 1:n structure. However, as a result of the way we select primary keys, some hierarchies can be m:n. The strict hierarchy structure is quite common, and several variations are possible.

*Pattern Variation 9a:
Institutional Hierarchy*

A corporation may be divided into companies, a company into business area, a business area into divisions, a division into departments, and so on. Hay (1996) discusses a similar structure. This hierarchical breakdown of institution can lead to a series of 1:n relationships (Figure 9a). If a strict hierarchy is not present, then the relationship, as illustrated in previous examples, can partially or completely be handled by m:n connectivity. Further, depending on the situation, levels can be bypassed.

In reviewing such applications, a database analyst needs to consider if the recursive pattern can be employed. However,

Figure 9a. Institutional hierarchy



in general, a recursive pattern is not advised since the attributes of a corporation will usually be different than the attributes of, say, a department.

Pattern Variation 9b: Container Hierarchy

In general, a container should represent a 1:n relationship. This is because, if one instance is physically contained in another, it should associate with one instance of the container (Figure 9b-1). If a bin has PDAs, each with its unique serial number, then the relationship is 1:n.

However, an interesting representation problem can arise sometimes. In theory, whether the relationship is 1:n or m:n depends on the identifier of the objects involved. For example, a warehouse can have several bins, and a bin can have sev-

eral items. Suppose there are four warehouses A, B, C, and D. If the bins are numbered A1 through A100, B1 through B100, and so on, then the relationship is 1:n. However, each warehouse may have bins numbered 1 through 100. Then, bin 1 may associate with any of the warehouses implying a m:n relationship. The better representation in this case is still 1:n, which can be shown by concatenating the primary key of Warehouse with the primary key of Bin (Figure 9b-2).

The logic of this approach is that the bin cannot really be identified by bin number and needs the warehouseID to identify it uniquely. If the container hierarchy expands to more levels, the primary key from the container can be carried over to the content to help identify the latter. This logic can be applied for any hierarchy representation.

Figure 9b-1. Container hierarchy



Figure 9b-2. Warehouse-Bin example



Another example of container hierarchy stems from the notion of aggregation (Booch, Rumbaugh & Jacobson, 1999). A car, for example, has components, such as chassis, cooling system, and electrical system. Each of these components further contain components and parts. This results in a strict hierarchy if discrete product numbers are used but may be m:n if generic product numbers are used.

*Pattern Variation 9c:
Geographic Hierarchy*

Geographic hierarchy generally represents a 1:n relationship (Figure 9c). However, this is possible only if the identifiers are globally unique (e.g., GPS coordinates) in the domain under consideration. This is occasionally not true in this domain, and we need to resort to the approach followed in the previous example. For example, a plot with a lot number is unique only within a development complex. If the domain contains many development complexes, then the relationship between lot number (say 10) and development complex is m:n, and we can follow the approach of converting the relationship to 1:n. Similarly, a city may be contained in a state, and, ostensibly, bear a 1:n relationship; yet, our common method of referring city by its name suggests an m:n relationship. For example, the city Bloomington is found in Indiana as well as in Minnesota. Further, it is possible that the same city name occurs more than once in the same state. Here, we can concatenate the zip code with city name to uniquely

identify the city. Alternatively, the geographical location can be identified using the latitude and longitude coordinates or the GPS coordinates.

Pattern 10: Plan

Planning involves developing a sequence of steps to be followed. Since it is fundamental to business activities, several data modeling pattern books (e.g., Fowler, 1997; Hay, 1996) have addressed it. Examples of plan include routing, project plan, battle plan, cooking recipe, batch job, student's degree plan, and so forth. Examples of plan steps include project plan tasks, recipe steps, batch job steps, courses to be enrolled, and so on. The relationships between plans and steps is shown in Figure 10. This is a simple version of the pattern. Complicated plans (e.g., battle plans) may have subplans in a recursive fashion. Further, each plan step can also be exploded in a recursive fashion. By combining the plan pattern with the recursive pattern, complicated plans and procedures can be addressed.

A common example of plan in business applications is routing. A routing is a list of the operations performed in a sequence to produce a product. Similarly, a recipe is a set of directions to prepare a dish. A procedure can be a sequential listing of steps to complete an activity. In such cases, the sequence of operations is an important characteristic. Hence, an attribute sequenceNo is used for the PlanStep.

Figure 9c. Geographic hierarchy

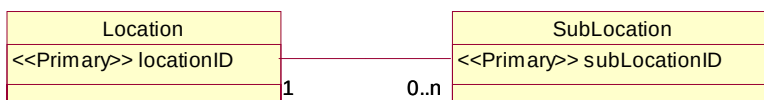


Figure 10. Plan pattern

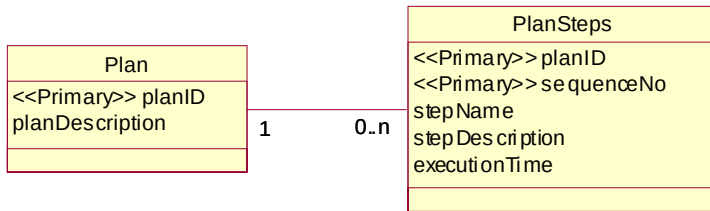
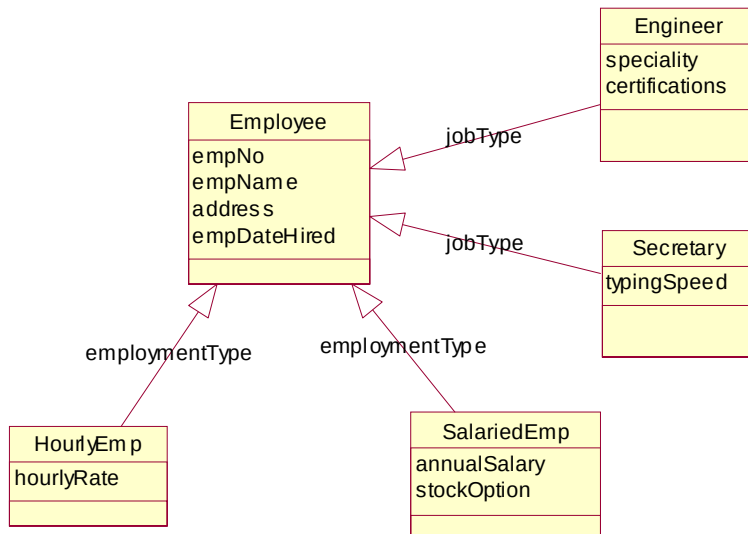


Figure 11. Examples of generalization pattern



Pattern 11: Generalization

The generalization structure, also known as the category structure as well as the subtype/supertype structure, is commonly covered in all textbooks (e.g., Hoffer, Prescott & McFadden, 2004), addressing data modeling. The generalization structure is a special kind of hierarchy, which represents the relationship and constraints between an object and the attributes that are specific to the object type (Smith & Smith, 1977). Since the primary key of the object and its type is the same, it results in kind of a one-one relationship. For example, an

entity **Employee** may have an attribute **EmployeeType**, which has values, such as **Engineer** and **Secretary**. Now, if **Engineer** has its own attributes, such as **Specialty** and **Certifications**, and **Secretary** has its own attributes, like **TypingSpeed**, **Engineer** and **Secretary** will be subtypes (or categories) of **Employee** (Figure 11). An entity can have more than one generalization hierarchy. Thus, in addition to a hierarchy based on **JobType**, an entity **Employee** may have another generalization hierarchy based on **EmploymentType** with **HourlyEmp** and **SalariedEmp** as the subtypes.

COMPARISON AMONG PATTERNS FROM THE DATA MODEL RESOURCES

For validation, the patterns described in the previous section were looked up in three data model resources: the reference models in the Scheer (1998) and Silverston, Inmon, and Graziono (1997) books, and the solutions obtained by solving the cases in the Whitlock et al. (2003) casebook, which is an academic source meant for students learning data modeling. Other academic casebooks, such as Easton and Easton (1995) and McFadden, Hoffer, and Srinivasan (1991), have similar cases and were, therefore, not considered. The immediate purpose of the validation was to discover if the proposed patterns were indeed found in the reference models with a reasonable frequency. A useful result of this exercise was to facilitate comparison among the three sources. Comparisons were to be made on the frequency of occurrence followed by observations on whether there was a preponderance of use of certain patterns. One of the goals of the paper was to conclude if casebooks reflected the structures in the more practitioner kind of books.

One of the issues that needed to be resolved was whether to consider all the reference models in the Scheer (1998) and the Silverston et al. (1997) books. There is a tendency in the books to build a complete data model for an application in typically two or three steps so that parts of an earlier model get repeated in the next. Since this would inflate the frequency of certain patterns, only the incremental portion of an application was considered. Let us say, a recursive structure is found in a bill of materials application. If the application is built in three stages, the recursive structure will

likely show up in all three diagrams. Counting it three times is likely to contaminate the results.

Another issue was to decide if all patterns were to be subjected to validation. It was decided that the data warehouse pattern would not be explicitly included for validation. Scheer (1998) and Silverston et al. (1997) each devote a full chapter to the data-warehousing topic. Since the data-warehousing pattern is frequently found in the discussion of the chapter, the data warehouse pattern was implicitly considered validated.

A more difficult question was how to treat the three transaction patterns, which are in a sense variations of the same underlying pattern. Scheer (1998) uses the time-based transaction pattern only but admits that this is only for clarity, and other ways of representing transactions may be employed. Silverston et al. (1997) does not typically use the time-based transaction pattern. An examination of treatise on patterns (e.g., Hay, 1996), database textbooks (e.g., Hoffer et al., 2004), and casebooks (e.g., Whitlock et al., 2003) indicated that generic transactions and discrete transactions are far more frequently employed. It was decided that any transaction pattern in the Scheer (1998) book be treated as a generic transaction occurrence and the same frequency be assumed for the discrete as well as the time-based transaction. The specific requirements of the application would determine the actual choice. Thus, the pattern was referred to as the transaction pattern.

The respective frequencies of each source are shown in Tables 1, 2, and 3. Table 1 shows the frequencies for the 48 applications found in the Scheer (1998) book; Table 2 shows the frequencies for the 46 applications found in the Silverston et al. (1997) book; and Table 3 shows the

frequencies for 8 cases from the academic-oriented Whitlock et al. (2003) book. The overall frequencies (see Table 4), despite some notable differences, appear some-

what similar between the two practitioner-oriented books—Scheer and Silverston. However, the overall frequencies of the academic- and student-oriented casebook

Table 1: Pattern frequencies found in the Scheer (1998) book

Figure	Application	Event	Type	GTrans	SubT	Rec	Hierch	Plan	Gen
B.I.03	Primary Requirement	0	0	0	0	0	0	1	0
B.I.19	Part Specialization	0	0	0	0	1	0	0	1
B.I.23,J	Complex Variant Formation	0	1	0	0	0	0	0	0
B.I.34	Order Definition	0	0	1	1	0	0	0	0
B.I.35	Requirements Planning	0	0	1	0	0	1	0	0
B.I.66	Requirements Management	0	0	0	0	0	0	0	1
B.I.102	Routing	0	0	0	0	0	0	1	1
B.I.103	Specializations of Routing	0	0	0	0	0	0	0	1
B.I.105	Scheduling & Capacity Planning	0	2	0	0	2	0	0	2
B.I.114	Interfaces to Acctg & Personnel	0	1	0	0	0	1	0	0
B.I.162	Order Calculation	0	1	0	0	0	0	0	0
B.I.164	Resource Specifications	0	0	0	0	0	0	0	1
B.I.165	Order Release	0	2	1	1	0	0	1	0
B.I.167	Order Interruption	1	2	0	0	0	0	0	0
B.I.170	Detailed Scheduling	3	0	0	0	0	0	0	1
B.I.184	Toolmaking	1	0	0	0	0	0	0	3
B.I.190	Warehouse	0	1	1	1	0	0	0	0
B.I.194	Materials Handling	0	1	0	0	0	0	0	0
B.I.205	Spare Parts	0	0	0	0	1	0	1	2
B.I.206	Maintenance Order	1	1	1	0	0	0	0	0
B.I.210	Factory Data Entry	1	0	0	0	0	0	0	1
B.I.221	CAM	0	0	0	0	0	0	0	1
B.II.07	Inbound Logistics	0	4	4	3	0	0	0	6
B.II.10	Purchase Order: Detailed	0	1	0	0	0	0	2	1
B.II.16	Purchase Order: Simplified	0	1	0	0	0	0	0	0
B.II.21	Outbound Logistics	0	5	4	3	0	0	0	6
B.II.25	Sales	0	3	0	0	1	0	2	1
B.II.28	Project Management	0	0	1	0	1	0	0	0
B.III.05	Human Resource Management	2	2	0	0	2	0	0	2
B.III.09	Personnel Accounting	1	2	0	0	0	0	0	0
B.III.13	Human Resource Planning	2	0	0	0	0	0	1	1
B.IV.02	Planning	3	3	0	0	1	0	1	2
C.II.06	Internal Data Sources	1	0	0	0	0	0	0	0
C.II.07	External Marketing Sources	0	2	0	0	2	0	0	0
C.II.15	Point Set Model	0	0	0	0	0	1	0	1
C.II.16	Surface Model	0	0	0	0	0	3	0	0
C.II.43	Development-Concurrent Costing	0	0	0	0	1	0	0	0
C.II.47	Disposal/Recycling routing	0	0	0	0	1	0	1	1
C.II.54	Integrated Product Data	0	0	0	0	0	0	0	0
C.III.04	Aggregation example	0	0	0	0	0	4	0	0
D.I.03	Account & Organizational Structures	0	2	0	0	3	4	0	6
D.I.06	Document Structure for Accounting	0	0	0	0	0	2	0	3
D.I.16	Cost & Income Categories	0	0	0	0	1	0	0	2
D.I.18	Cost Centers	0	1	0	0	2	0	0	1
D.I.22	Cost Objectives	0	0	0	0	0	0	0	1
D.I.24	Overhead Cost Allocation	0	1	0	0	0	2	0	1
D.I.30	Product Costing	0	0	0	0	1	0	0	0
D.I.43	Cost Accounting	0	0	0	0	0	0	0	2
Total		16	39	14	9	20	18	11	52

Table 2. Pattern frequencies found in Silverston et al. (1997)

Figure	Application	Event	Type	GTrans	SubT	Rec	Hierch	Plan	Gen
2.1	Organizational Definition	0	0	0	0	0	0	0	1
2.2	Person Definition	0	0	0	0	0	0	0	1
2.3	Party Definition	1	1	0	0	0	0	0	1
2.6	Address Definition	1	1	0	0	0	3	0	0
2.7	Contact Mechanism	1	1	0	0	0	0	0	1
2.8	Contact Information	1	4	0	0	0	0	0	0
3.1	Basic Product Definition	2	4	0	0	1	0	0	1
3.2	Product Supplier	1	2	0	0	0	0	0	1
3.3	Inventory Item Storage	0	1	0	0	0	2	0	0
3.4	Standard Product Pricing	1	1	0	0	0	0	0	1
3.5	Estimated Product Cost	0	1	0	0	0	0	0	0
3.6	Product Components	2	0	0	0	0	0	0	1
4.1	Ordering Products	0	0	2	0	0	0	0	0
4.3	Order Header	0	2	0	0	0	0	0	0
4.4	Order Line Item	0	3	0	0	0	0	0	2
4.6	Person Roles for Order	1	1	0	0	0	0	0	0
4.7	Requisition Definition	1	2	1	0	0	0	0	0
4.8	Request Definition	0	0	1	0	0	0	0	1
4.9	Quote Definition	0	0	1	1	0	0	0	1
4.10	Person Roles for Reqs. & Quotes	0	2	0	0	0	0	0	0
4.11	Agreement Definition	1	1	1	1	0	0	0	1
5.1	Shipment Definition	1	1	1	1	0	0	0	1
5.2	Ship method and Ship Vehicle	0	2	0	0	0	0	0	1
5.3	Invoice Definition	1	3	1	0	1	0	0	1
5.4	Shipment and Invoice Association	0	0	1	1	0	0	0	0
5.7	Invoice Billing	1	0	0	0	0	0	0	0
5.8	Delivery Model	0	0	0	0	0	1	0	0
6.1	Work Order Definition	0	2	1	0	0	0	0	0
6.2	Work Order Role	0	1	0	0	0	0	0	0
6.3	Work Effort Generation	0	1	1	1	0	0	0	1
6.4	Work Task Definition	0	2	0	0	1	0	1	0
6.5	Work Effort and Party Allocation	1	1	0	0	0	0	0	0
6.6	Work Task Assignment	1	2	0	0	0	0	0	0
6.8	Fixed Asset Assignment	0	2	0	0	0	0	0	1
6.11	Work Order Invoicing	0	1	1	0	0	0	0	0
7.1	Chart of Accounts	0	2	0	0	0	0	0	0
7.2	Accounting Transaction Definition	0	2	1	0	1	0	0	3
7.3	Asset Depreciation	0	0	0	0	0	0	0	1
7.4	Budget Definition	1	3	1	0	0	0	0	0
8.1	Position Definition	1	2	0	0	0	0	0	0
8.2	Position Type Definition	1	1	0	0	0	0	0	0
8.3	Position Reporting Relationships	1	0	0	0	0	0	0	0
8.4	Position Fulfillment	1	1	0	0	0	0	0	1
8.5	Salary Determination and History	2	2	0	0	0	0	0	0
8.6	Benefits Definition and Tracking	0	2	0	0	0	0	0	0
8.7	Payroll Information	1	2	1	0	0	0	0	0
Total		26	62	15	5	4	6	1	23

Table 3. Pattern frequencies found in the Whitlock et al. (2003) casebook

Case	Case Name	Event	Type	GTrans	SubT	Rec	Hierch	Plan	Gen
A	Wright Technical	0	1	2	0	1	0	0	0
B	Fit-4-Life Fitness	0	2	4	1	0	0	0	0
C	FirstCare Medical	0	1	3	2	0	0	0	1
E	NCAA Basketball	2	0	1	0	0	0	0	0
F	Cellibini Bicycle	0	2	2	0	1	1	0	1
G	Conference	2	1	3	2	0	0	0	1
H	Rose Garden	0	0	2	1	0	0	0	1
I	Westcoast Floral	0	0	4	2	0	0	0	1
Total		4	7	21	8	2	1	0	5

Table 4. Overall numerical and percentage pattern frequency

Source	Event	Type	Gtrans	SubT	Rec	Hierch	Plan	Gen
Scheer	16	39	14	9	20	18	11	52
Scheer Percent	8.9	21.8	7.8	5.0	11.2	10.1	6.1	29.1
Silverston	26	62	15	5	4	6	1	23
Silverston Percent	18.3	43.7	10.6	3.5	2.8	4.2	0	16.2
Whitlock	4	7	21	8	2	1	0	5
Whitlock Percent	8.3	14.6	43.7	16.7	4.2	2.1	0	10.4

show a distinct departure from the other two.

The first difference is that the leading patterns of the data sources are different. In the Scheer (1998) source, the leading pattern is generalization; in the Silverston et al. (1997) source, the leading pattern is type; and in the Whitlock et al. (2003) source, the leading pattern is the transaction. In each of the latter two, the leading pattern accounts for more than 40% of the total number. There is some similarity between the two practitioner sources: the event, the type, and the generalization patterns were generally the dominant ones. But, these two sources had different patterns distribution than that of the academic source.

Table 4 can be treated as a contingency table and statistically analyzed using the Chi-square test. The resulting analysis

yields a Chi-square of 104.4 with 14 degrees of freedom, and a p-value less than 0.001. Since the Scheer and Silverston entries are based on practitioner-oriented sources while the Whitlock entries are based on an academic-oriented source, the Scheer and Silverston entries can be pooled and contrasted with the Whitlock entry. This results in a Chi-square of 60.0 with 7 degrees of freedom, which again leads to a p-value less than 0.001.

So what can we conclude? It seems that cases written for the database student are adequate but not accurate representatives of the structures found in real-world situations. Cases for students revolve around transactions. This is not such a bad situation since the core of business is transactions, but scenarios that also better reflect other structures may serve the student. Table 4 reveals that all structures other

than transactions (and subsequent transactions) are underrepresented in the academic source. Specifically, the generalization structure seems to require the most attention. This is probably because relational DBMS, which dominate the industry, have not implemented generalization until recently, and instructors may not be keen to teach structures whose implementation cannot be shown directly.

Another structure that is underrepresented in the academic source is the type structure. This structure usually has a 1:n relationship. For example, product and product type have a 1:n relationship with product on the many side. From an academic perspective, this structure is simple, and repeating it too many times will not enhance student learning. Still, it would be better to have more examples for the type structure in academic cases. In the real world, the structure is indispensable for reporting purposes.

The hierarchy structure is well represented in the Scheer (1998) source but not in the other two sources. This is so because Scheer (1998) provides good coverage on aggregation structures, and the other two do not; Silverston et al. (1997) does not address aggregation structures, while Whitlock et al. (2003) may have purposely ignored it because the aggregation structure is not easily implemented by contemporary DBMS. However, other examples of hierarchy did not show up in any of the three sources. The hierarchy pattern needs further scrutiny and may have to be renamed. Siau (2004) identifies part-whole (i.e., aggregation) as one of the five families of semantic relationships.

The absence of plan structure in the academic source is of concern. Perhaps, this is an artifact of the sample. Evaluation of another academic source is required to verify this observation.

The analysis of the data model sources provides a robust validation of the proposed patterns. If the association structure, commonly found in data model sources and not regarded as a pattern, is ignored, the remaining data structures can be explained by the patterns listed in the paper.

CONCLUSION

This paper extracts eleven data modeling patterns from three books on data modeling patterns, as well as textbooks and casebooks. The patterns are entity event, entity type, generic transaction, discrete transaction, time-based transaction, data warehouse, subsequent transaction, recursive, strict hierarchy, plan, and generalization. The patterns are then validated from books that have a large number of reference data models. The validation suggests that these patterns are, indeed, commonly found in business applications.

Promising empirical work in the area of pattern reuse has already commenced. Batra and Wishart (2004) have shown that patterns may lead to better designer performance for problems of medium complexity. Purao, Storey, and Han (2003) have empirically tested a prototype that incorporates learning mechanisms employing usage history and found that it improves reuse-based conceptual designs over naïve approaches.

The suggested patterns are likely to be useful in training and in developing expert tools. Patterns should be considered as another method to approach data modeling, not as a replacement for all other methods. The other methods include rules and heuristics (Batra & Zanakakis, 1994) and normalization (Date, 1999). Perhaps a suitable combination of these approaches may lead to an effective training and use strategy.

ACKNOWLEDGMENT

The author would like to thank the anonymous reviewers for their critical, meticulous, and helpful comments.

REFERENCES

- Anderson, J.R. (1985). *Cognitive psychology and its implications*. New York: W.H. Freeman.
- Anderson, J.R. (1996). *The architecture of cognition*. Cambridge, MA: Harvard University Press.
- Batra, D., & Wishart, N.A. (2004). Comparing a rule-based approach with a pattern-based approach under varying task complexity in conceptual data modeling. *International Journal of Human Computer Interaction*, 61(4), 397-419.
- Batra, D., & Zanakis, S.H. (1994). A conceptual database design methodology based on rules and heuristics. *European Journal of Information Systems*, 3(3), 228-239.
- Booch, G., Rumbaugh, J., & Jacobson, I. (1999). *The unified modeling language user guide*. Upper Saddle River, NJ: Prentice Hall.
- Chen, P.P. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems*, 1(1), 9-36.
- Chi, M.T.H., Glaser, R., & Farr, M.J. (1988). *The nature of expertise*. Hillsdale, NJ: Lawrence Erlbaum.
- Coad, P. (1997). *Object models: Strategies, patterns and applications*. Upper Saddle River, NJ: Prentice Hall.
- Codd, E.F. (1970). A relational model of data for large shared banks. *Communications of the ACM*, 13(6), 377-387.
- Date, C.J. (1999). *An introduction to database systems*. Reading, MA: Addison-Wesley.
- Easton, A., & Easton, G. (1996). *Cases for modern systems analysis and design*. Reading, MA: Addison-Wesley.
- Fowler, M. (1997) *Analysis patterns: Reusable object models*. Reading, MA: Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Reading, MA: Addison-Wesley.
- Gemino, A., & Wand, Y. (2004). Dimensions in experimental evaluation of conceptual modeling techniques. *Proceedings of the 3rd Symposium on Systems Analysis and Design Research, St. John's*, (pp. 1-20).
- George, J.F., Batra, D., Valacich J.S., & Hoffer, J.A. (2004). *Object-oriented systems analysis and design*. Upper Saddle River, NJ: Pearson Prentice Hall.
- Gray, P., & Watson, H.J. (1998). *Decision support in the data warehouse*. Upper Saddle River, NJ: Prentice Hall.
- Greeno, J.G. (1989). Situations, mental models, and generative knowledge. In D. Klahr & K. Kotovsky (Eds.), *Complex information processing: The impact of Herbert A. Simon*. Hillsdale, NJ: Lawrence Erlbaum.
- Hay, D.C. (1996). *Data model patterns: conventions of thought*. Dorset House Publishing Company.
- Hoffer, J.A., Prescott M.B., & McFadden, F.R. (2004) *Modern database management* (7th ed.). NJ: Prentice-Hall.
- Kotovsky, K., & Fallside, D. (1989). Representation and Transfer in Problem Solving. In D. Klahr & K. Kotovsky (Eds.), *Complex information processing: The impact of Herbert A. Simon*. Hillsdale, NJ: Lawrence Erlbaum.
- McFadden, F.R., Hoffer, J.A., & Srinivasan, A. (1991). *Casebook for database management*. Redwood City, CA: Ben-

- jamin Cummings.
- Metsker, S.J. (2002). *Design patterns Java workbook*. Reading, MA: Addison-Wesley.
- Newell, A., & Simon, H. (1972). *Human problem solving*. Englewood Cliffs, NJ: Prentice Hall.
- Purao, S., Storey, V., & Han, T. (2003). Improving reuse-based design: Augmenting analysis patterns reuse with learning. *Information Systems Research*, 14(2), 269-290.
- Reeves, W.W. (1996). *Cognition and complexity: The cognitive science of managing complexity*. London: Scarecrow Press.
- Reynolds, A.G., & Flagg, P.W. (1977). *Cognitive psychology*. Cambridge, MA: Winthrop Publishers.
- Rosenberg, D. (1999). *Use case driven object modeling with UML*. Reading, MA: Addison-Wesley.
- Rumbaugh, J., Jacobson, I., & Booch, G. (1999). *The unified modeling language reference*. Reading, MA: Addison-Wesley.
- Scheer, A.W. (1998). *Business process engineering: Reference models for industrial enterprises*. New York: Springer-Verlag.
- Scott, K. (2001). *UML explained*. Upper Saddle River, NJ: Addison-Wesley.
- Shah, D., & Slaughter, S. (2001). Data modeling and UML. In K. Siau & T. Halpin (Eds.), *Unified modeling language: Systems analysis, design and development issues*. Hershey, PA: Idea Group.
- Siau, K. (2004). Relationship construct in modeling information systems: Identifying relationships based on relational element theory. *Journal of Database Management*, 15(3), i-v.
- Silverston, L., Inmon, W.H., & Graziono, K. (1997). *The data model resource book: A library of logical data and data warehouse designs*. New York: John Wiley & Sons.
- Smith, J., & Smith, D. (1977) Database abstractions: Aggregation and generalization. *ACM Transactions on Database Systems*, 2(2), 105-133.
- Whitlock, W., Nelson, K., & Papp, R. (2003). *Modern database management casebook* (6th ed.). Upper Saddle River, NJ: Pearson Education.
- Weber, R. (2003). Conceptual modeling and ontology: Possibility and pitfalls. *Journal of Database Management*, 14(3), 1-20.

Dinesh Batra is a professor in the College of Business Administration at the Florida International University. His publications have appeared in Management Science, Communications of the ACM, Journal of MIS, Journal of Database Management, Decision Support Systems, Data Base, European Journal of Information Systems, International Journal of Human Computer Studies, Computers and Operations Research, Information and Management, and others. His research interests focus on usability issues in database design and use, systems development methodologies, and knowledge-based systems. He is the president of the AIS Special Interest Group on Systems Analysis & Design (SIGSAND).